

PROJET ROVER

SPRINT 3 : Navigation/API

L'objectif de ce sprint est de développer des fonctions de navigation permettant au Rover de se déplacer dans différentes directions, d'exécuter des trajectoires simples et répétables et, éventuellement, d'éviter des obstacles en s'appuyant sur les encodeurs et les fonctions de contrôle moteur.

Vous utiliserez les fonctionnalités du driver moteur du Rover (Sprint 2) pour développer progressivement un module complet de navigation.

1 Correction du SPRINT 2

1.1 Fonction de contrôle de la vitesse

Lors du sprint 2, vous avez implémenté un algorithme de contrôle des moteurs basé sur une boucle fermée : la commande envoyée au moteur est continuellement ajustée en fonction des mesures fournies par l'encodeur. Ce type de régulation utilise généralement des algorithmes PI (Proportionnel-Intégral) ou PID (Proportionnel-Intégral-Dérivé), que l'on peut concevoir avec des outils comme Matlab/Simulink ou implémenter directement après avoir déterminé leurs paramètres.

Dans le sprint 3, vous utiliserez la fonction fournie dans l'annexe (`motor_driver_lib.py`) pour définir la vitesse des moteurs. Le driver appliquera automatiquement la régulation en boucle fermée, ce qui vous permettra de vous concentrer exclusivement sur la logique de navigation :

```
control_motor_speeds(m1, m2, m3, m4)
```

2 Objectifs du Sprint 3

2.1 Premières fonctions de navigation

Au cours de ce Sprint 3, vous allez développer différentes fonctions de contrôle et de navigation du rover. Vous disposerez d'une fonction permettant de commander la vitesse de chaque moteur (cf. partie 1.1 et correction) et de lire les valeurs de encodeurs.

Objectifs :

À partir de ces fonctions, vous devrez implémenter les fonctions :

- avancer d'une distance donnée : `avance(distance, vitesse)`

- tourner d'un angle donné : `tourne(degrees, vitesse)`

Étapes à suivre :

Pour cela, vous devez d'abord **calibrer** votre robot afin d'obtenir la relation entre les **ticks d'encodeur** et :

- la distance parcourue
- l'angle de rotation réalisé

Utilisez ensuite la fonction `controle_vitesse(v1, v2, v3, v4)` pour faire avancer le rover sur la distance souhaitée, en vous basant sur le nombre de ticks à atteindre.

2.1.1 Déplacement en ligne droite

Déterminer précisément la relation entre la distance parcourue par le rover et le nombre de ticks mesurés par les encodeurs, afin d'implémenter une fonction `avance(distance, vitesse)` reposant sur une distance réelle.

A. Détermination de la relation ticks ↔ distance

1. **Mesure des ticks via les encodeurs** : Vous disposez de la fonction `read_total_encoder_counts()` qui retourne, pour chaque roue, le nombre total de ticks mesurés depuis la dernière remise à zéro. Utilisez cette fonction pour récupérer le nombre de ticks d'un moteur au cours des expériences.
2. **Réalisation des expériences de déplacement** :
 - Choisir une vitesse constante (ex. 30 %), et faire avancer le rover en ligne droite pendant un temps déterminé (ex. 2 s).
 - Relever le nombre total de ticks mesurés et l'enregistrer dans un log (ticks, vitesse, durée).
 - Mesurer la distance réellement parcourue (à l'aide d'une règle ou d'un mètre)
 - Répéter l'expérience plusieurs fois, éventuellement avec différentes vitesses, afin d'obtenir des données fiables pour la calibration.

3. Calcul de la relation ticks → distance

Tracez une régression linéaire avec :

- axe X : nombre de ticks
- axe Y : distance parcourue

Vous obtiendrez une équation du type : **$distance = a \times ticks + b$**

En général, avec des mesures propres, $b \approx 0$, ce qui permet d'utiliser :

$$CM_PAR_TICK = a$$

$$TICKS_PAR_CM = \frac{1}{a}$$

Ces coefficients serviront dans la fonction `avance()`.

B. Implémentation de la fonction `avance(distance, vitesse)`

Utiliser les paramètres de calibration obtenus pour programmer une fonction `avance(distance, vitesse)` capable de faire parcourir au rover une distance donnée en convertissant cette distance en un nombre de ticks cible, puis en contrôlant

les moteurs de façon continue jusqu'à ce que les ticks mesurés atteignent la valeur correspondante et permettent d'arrêter le déplacement avec précision.

2.1.2 Rotation du rover

Pour que le rover puisse tourner d'un angle précis, il est nécessaire d'établir une relation entre le nombre de ticks générés par les roues lors d'une rotation sur place et l'angle réellement parcouru. Cette calibration permet ensuite de commander des rotations exactes en degrés.

Pour réaliser un pivot, il suffit de faire tourner les roues d'un côté vers l'avant et celles de l'autre côté vers l'arrière. Par exemple, roues droites en avant et roues gauches en arrière provoquent une rotation sur place.

A. Calibration de la rotation

- Faire tourner le rover sur lui-même à vitesse constante (ex. roues droites en avant et roues gauches en arrière).
- Commander une rotation continue (ex. : $v1 = v2 = +V$ et $v3 = v4 = -V$).
- Relever le nombre total de ticks mesurés et les enregistrer dans un log.
- Mesurer l'angle réellement parcouru (ex. une rotation complète = 360°).
- Dédire la relation entre ticks et angle, par exemple :

$$angle = k \times ticks$$

ou

$$ticks_par_degre = \frac{ticks_pour_360}{360}$$

B. Implémentation de la fonction `tourne(angle, vitesse)`

La fonction `tourne(angle, vitesse)` doit convertir l'angle demandé en ticks-cible, remettre les compteurs d'encodeurs à zéro, puis piloter les roues en sens opposés jusqu'à ce que la rotation atteigne le nombre de ticks attendu.

Rappel

Paramètre	Signification	Valeur typique	Comment obtenir
TICKS_PAR_CM	Nombre de ticks par centimètre	10-20 ticks/cm	Avancer ~50 cm et mesurer les ticks
TICKS_PAR_DEGRE	Nombre de ticks par degré de rotation	1-3 ticks/°	Faire tourner 360° puis diviser les ticks par 360

2.2 Navigation

Définition de la fonction `navigue` :

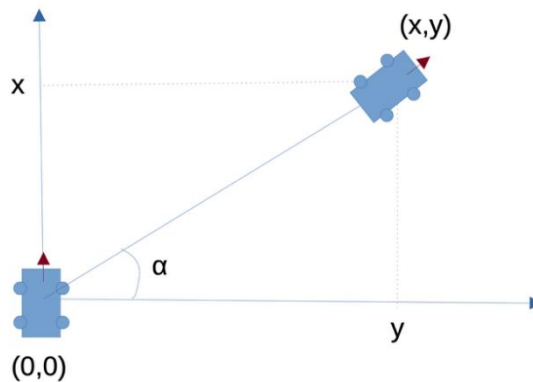
Grace aux deux fonctions précédentes, vous pouvez désormais définir une nouvelle fonction :

```
navigue(x, y, vitesse, angle)
```

Celle-ci permet de donner une destination au rover, exprimée en coordonnées (x,y) en centimètres, relativement à sa position actuelle.

L'approche utilisée ici correspond à une navigation open-loop basée sur :

- orienter le rover vers la cible à l'aide de `tourne()`
- puis parcourir la distance correspondante à l'aide de `avance()`



Mesurer l'erreur :

Lors de l'exécution des fonctions `avance()` et `tourne()`, il est important de mesurer l'écart entre le déplacement attendu et le déplacement réel observé.

Cette erreur, due principalement au glissement des roues et aux imperfections mécaniques, vous permet d'évaluer la précision de votre calibration, d'identifier les limites de la navigation open-loop et d'améliorer la fiabilité de vos déplacements.

2.3 Navigation avancée (au choix/optionnelle)

Cette partie propose plusieurs améliorations possibles pour la suite du projet.

Vous pouvez choisir celle(s) que vous souhaitez travailler.

Extensions possibles :

A. Navigation avec détection et évitement d'obstacles

Vous allez maintenant définir une fonction `navigue_obstacle(x, y, vitesse, angle)` qui reprend le principe de la navigation précédente, mais en intégrant l'évitement d'obstacles.

Pour cela, vous devez utiliser le capteur de distance afin de détecter les obstacles sur le trajet. Il faudra créer une fonction qui lit les données de ce capteur, puis concevoir une nouvelle version de la fonction de navigation capable de réagir en conséquence.

B. Exécuter des formes géométriques

Implémenter des fonctions de navigation avancées de type `arc_de_cercle(rayon, distance)` ou `carre(coté)` etc.

C. Suivi de trajectoires complexes

D. Porter le code en orienté objet

E. Bibliographie sur la cinématique inverse (optionnelle)

L'approche développée dans ce sprint montre ses limites en termes de précision dans certaines situations. Pour améliorer la navigation, l'une des solutions les plus courantes et fiables consiste à utiliser une boucle fermée avec asservissement en position ainsi qu'un algorithme de cinématique inverse.

Effectuez une recherche documentaire sur ces deux concepts puis réfléchissez à comment les intégrer pour améliorer la fonction `navigue(x,y,vitesse)`.

3 Modalités d'évaluation du projet – Sprint 3

Le projet sera évalué lors d'une soutenance orale. La note finale (sur 20) sera attribuée selon la grille suivante :

1. Développement (10 points)

Critère	Points	Points
Fonction avance(distance, vitesse) et tourne(angle, vitesse)	2	2
Fonction de navigation goto / navigue(position, vitesse, angle)	2	2
Mesure et analyse de la précision des déplacements	2	2
Fonctions supplémentaires / extensions (navigation avancée, etc.)	4	4

2. Présentation (10 points)

Critère	Points
Respect de la durée (10 minutes)	2
Nombre de diapositives adapté (maximum 8 slides)	2
Qualité des schémas et explications	2
Clarté des réponses aux questions	4

La note finale du projet est la somme des points obtenus dans les parties **Développement** et **Présentation**, pour un total de **20 points**.

4 ANNEXE 1

Solution du Sprint 2

Motor_driver_lib.py

```
import struct
import time

i2c = None # Global I2C object
MOTOR_I2C_ADDRESS = 0x26

TICKS_PER_REVOLUTION = 2340
SAMPLE_INTERVAL = 0.01 # seconds

import os

LOG_DIR = "logs" # création du dossier de stockage des logs
LOG_FILE = LOG_DIR + "/speed_log0.csv" # Nom du fichier des logs dans le dossier

REG = {
    'TYPE': 0x01,
    'DEADZONE': 0x02,
    'PULSE_PER_LINE': 0x03,
    'PHASE_COUNT': 0x04,
    'WHEEL_DIAMETER': 0x05,
    'SPEED': 0x06,
    'PWM': 0x07,
    'ENCODER_DELTA': [0x10, 0x11, 0x12, 0x13],
    'ENCODER_HIGH': [0x20, 0x22, 0x24, 0x26],
    'ENCODER_LOW': [0x21, 0x23, 0x25, 0x27],
}

def init_logging():
    if LOG_DIR not in os.listdir():
        os.mkdir(LOG_DIR)
    with open(LOG_FILE, "w") as f:
        f.write("time_ms, V1_rpm, V2_rpm, V3_rpm, V4_rpm\n")

# Ajoute une ligne de mesure dans le fichier CSV :
# Le temps en ms
# La vitesses des 4 roues en rpm
def append_log(temps_ms, V1, V2, V3, V4):
    with open(LOG_FILE, "a") as f:
        f.write(f"{temps_ms},{V1},{V2},{V3},{V4}\n")

def init_i2c(i2c_instance):
    global i2c
    i2c = i2c_instance

def write_register(reg, data_bytes):
```

```

i2c.writeto_mem(MOTOR_I2C_ADDRESS, reg, bytearray(data_bytes))

def read_register(reg, length):
    return i2c.readfrom_mem(MOTOR_I2C_ADDRESS, reg, length)

def float_to_bytes(value):
    return struct.pack('<f', value)

# -----
# Motor Configuration for type 3 TT DC Motor
# -----
def set_motor_parameters():
    write_register(REG['TYPE'], [3]) # Motor type 3: TT encoder
motor
    time.sleep(0.05)
    write_register(REG['PHASE_COUNT'], [0x00, 45]) # 45 phase count
    time.sleep(0.05)
    write_register(REG['PULSE_PER_LINE'], [0x00, 13]) # 13 pulse/line
    time.sleep(0.05)
    write_register(REG['WHEEL_DIAMETER'], list(float_to_bytes(65.0))) # 65 mm diameter
    time.sleep(0.05)
    write_register(REG['DEADZONE'], [0x06, 0x40]) # 1600 (0x0640) deadzone
    time.sleep(0.05)

# -----
# Motor Control
# -----
def control_motor_pwm(p1, p2, p3, p4):
    pwms = []
    for v in (p1, p2, p3, p4):
        pwms.extend([(v >> 8) & 0xFF, v & 0xFF])
    write_register(REG['PWM'], pwms)

def control_motor_speeds(m1, m2, m3, m4):
    speeds = []
    for v in (m1, m2, m3, m4):
        speeds.extend([(v >> 8) & 0xFF, v & 0xFF])
    write_register(REG['SPEED'], speeds)

# -----
# Encoder Readings
# -----
def read_encoder_deltas():
    ticks = []
    for reg in REG['ENCODER_DELTA']:
        buf = read_register(reg, 2)
        value = (buf[0] << 8) | buf[1]
        if value & 0x8000:
            value -= 0x10000
        ticks.append(value)

```

```

    return ticks

def read_total_encoder_counts():
    counts = []
    for high_reg, low_reg in zip(REG['ENCODER_HIGH'], REG['ENCODER_LOW']):
        high = read_register(high_reg, 2)
        low = read_register(low_reg, 2)
        value = ((high[0] << 8) | high[1]) << 16 | ((low[0] << 8) | low[1])
        if value & 0x80000000:
            value -= 0x100000000
        counts.append(value)
    return counts

# -----
# RPM Calculation
# -----

def calculate_rpm(ticks, ticks_per_rev=TICKS_PER_REVOLUTION, interval=SAMPLE_INTERVAL):
    return (ticks / ticks_per_rev) * 60 / interval

def main():
    from machine import I2C, Pin
    import time
    print("debut")

    i2c = I2C(0, scl=Pin(5), sda=Pin(4), freq=400000)
    init_i2c(i2c)
    set_motor_parameters()
    print(read_total_encoder_counts())
    sp = 400
    control_motor_speeds(sp, sp, sp, sp)
    time.sleep(1)
    control_motor_speeds(0, 0, 0, 0)
    print(read_total_encoder_counts())

    print("fin")

main()

```